

1. Introduction

The purpose of this document is to describe the major engineering and architectural decisions made throughout the development of the Voice-based Retrieval-Augmented Generation (RAG) Assistant.

Rather than selecting technologies based solely on popularity, each decision was evaluated according to multiple engineering criteria including maintainability, scalability, implementation complexity, retrieval quality, latency, deployment flexibility, cost, and suitability for the assignment. Where multiple alternatives were considered, trade-offs were analysed before the final solution was selected.

2. System Architecture Decisions

3 Technology Selection

4. RAG Design Decisions

5. Testing Decisions

6. Performance Optimisation Decisions

6.1 Optimisation Objectives

After completing the initial implementation of the Voice-based RAG system, the focus shifted from functional correctness to improving the overall user experience. Several performance bottlenecks were identified during testing, particularly in response latency, retrieval efficiency, resource consumption, and large document processing.

The optimisation process aimed to achieve the following objectives:

- Reduce end-to-end response latency.

- Improve retrieval efficiency.
- Maintain retrieval quality while reducing computational cost.
- Improve scalability for larger PDF documents.
- Provide deployment flexibility for different hardware environments.

6.2 Retrieval Pipeline Optimisation

Design Considerations

The initial retrieval pipeline included several advanced retrieval techniques intended to maximise retrieval quality. However, testing showed that enabling every optimisation for every request significantly increased response time without providing proportional improvements in answer quality.

Therefore, the retrieval pipeline was redesigned to prioritise response speed during normal operation while reserving more computationally expensive techniques for difficult queries.

Final Decision

The standard retrieval pipeline focuses on low latency, while advanced retrieval techniques such as Query Expansion and Agent-based RAG are only activated through the fallback mechanism when the initial retrieval fails to obtain sufficiently relevant results. This approach provides a better balance between user experience and retrieval quality.

6.3 Self-RAG Optimisation

Problem

Self-RAG was initially investigated as a method for improving answer quality by performing additional reasoning and retrieval verification before generating the final response.

Evaluation

Although Self-RAG improved retrieval robustness in some scenarios, testing indicated that it introduced multiple additional LLM calls, increasing overall response latency while providing only marginal improvements in retrieval quality.

Final Decision

Self-RAG was removed from the final architecture.

Reasons:

- Eliminates approximately two additional LLM calls.
- Significantly reduces response latency.
- Simplifies the retrieval pipeline.
- Reranking already provides sufficient retrieval quality for this assignment.

6.4 Agent RAG Optimisation

Problem

Agent-based RAG was originally considered for automatically selecting different retrieval strategies according to user intent.

Evaluation

Although the agent improved retrieval flexibility, it introduced additional reasoning steps before retrieval, resulting in higher latency and additional API cost.

Final Decision

Instead of removing Agent RAG entirely, it was incorporated into the fallback retrieval mechanism.

This design allows the normal retrieval workflow to remain lightweight while still providing more sophisticated retrieval capabilities when required by complex or ambiguous user queries.

6.5 Query Expansion Optimisation

Problem

Query expansion improves retrieval quality by generating richer search queries before retrieval.

However, generating expanded queries requires additional LLM inference, increasing response latency.

Final Decision

Query expansion was removed from the default retrieval pipeline and incorporated into the fallback mechanism together with Agent-based RAG.

This design allows the majority of user requests to complete with lower latency while maintaining the ability to improve retrieval quality when the initial retrieval is insufficient.

6.6 Whisper Deployment Optimisation

Design Considerations

The Faster Whisper model provides excellent recognition accuracy but its inference speed depends heavily on available hardware.

Different deployment environments may have different computational capabilities.

Final Decision

The system supports both local GPU inference and cloud-based inference.

This provides deployment flexibility:

- Local GPU for faster inference on capable machines.
- Cloud execution for devices without sufficient computational resources.

6.7 Embedding Optimisation

Design Considerations

Embedding generation represents a significant portion of document processing time, particularly for large PDF documents.

Both local embedding models and cloud-based embedding services were evaluated.

Evaluation

Cloud-based embedding services provide lower memory consumption and faster inference, while local embedding models provide lower operating cost and greater deployment independence.

Final Decision

The embedding implementation remains configurable.

The current implementation retains local embedding while allowing cloud-based embedding to be adopted through configuration when required by deployment constraints.

6.8 Large PDF Processing Optimisation

Problem

Testing with large PDF documents showed that processing all document chunks simultaneously resulted in excessive memory usage and slower processing.

Final Decision

Instead of processing the entire document in a single operation, PDF chunks are processed in batches.

This batching strategy:

- Reduces memory consumption.
- Improves stability for large documents.
- Maintains document processing efficiency.

6.9 Reranker Optimisation

Design Considerations

Although reranking significantly improves retrieval precision, it also increases overall response latency.

Final Decision

The system provides flexibility for different deployment scenarios by allowing:

- Reranker execution on GPU when available.
- The reranking stage to be skipped when faster responses are preferred.

This enables users to balance retrieval quality and response speed according to practical requirements.

6.10 Audio Streaming Optimisation

Problem

Testing indicated that waiting for the entire synthesized speech before playback increased perceived response latency.

Final Decision

WebSocket-based streaming was selected to deliver synthesized audio progressively.

This allows audio playback to begin before the complete speech synthesis process has finished, improving the perceived responsiveness of the application.

6.11 Prompt Optimisation

Design Considerations

Prompt engineering was refined to improve the quality and readability of generated responses.

Several issues were identified during testing, including unnecessary special characters, filler words, and insufficient conversational context.

Final Decision

The response generation process was refined by:

- Removing unnecessary emojis and special characters.
- Removing filler words before passing the query to the Large Language Model.
- Improving prompts to better utilise conversation history.
- Adding document citations to improve response traceability.

6.12 Audio Visualizer

Design Considerations

Users of the front-end need to know whether they are producing sound; this feedback enables them to better gauge the volume of their voice and adjust it accordingly.

Final Decision

- Takes levels array and renders vertical bars
- Each bar height maps to a frequency bin

7. GPU Runtime Evaluation

Objective

Before performing further retrieval optimisation, I wanted to evaluate the system under a more realistic deployment environment where GPU acceleration was available. Since the final application is intended to provide responsive voice interactions, measuring the retrieval performance on CPU alone would not accurately reflect the system's practical capability.

Therefore, the backend server was temporarily migrated to **Google Colab**, allowing GPU acceleration to be used for computationally intensive components such as speech recognition, embedding generation, and reranking. The objective was to determine whether the retrieval pipeline itself could achieve acceptable performance when hardware limitations were reduced.

Performance Evaluation

During testing, the Google Gemini API frequently reached its free-tier rate limits, making it difficult to obtain stable latency measurements. To avoid the API bottleneck affecting the evaluation, the LLM was temporarily replaced with the **DeepSeek API**, which provided more stable request throughput for performance benchmarking.

After migrating the backend to a GPU-enabled environment, the overall retrieval latency was significantly reduced. This indicated that the major performance bottlenecks were no longer dominated by hardware limitations but by the retrieval strategy itself.

Latency Result:

```

=====
LATENCY SUMMARY (excluding first query - cold start)
=====

Metric                Voice Query
-----
Min Latency            2.052s
Max Latency            7.438s
Avg Latency            3.340s
P50 Latency            2.668s
P95 Latency            7.438s

TTS Latency            1.788s (avg)

=====
PERFORMANCE TARGETS
=====
[PASS] Voice query P95 < 8s
[PASS] TTS avg < 2s

```

Loading Result:

```

=====
LOAD TEST SUMMARY
=====

Users    Queries    Success%    Throughput    Avg Lat    P95 Lat
-----
1         5           100.0       0.32          2.500     3.552
5         25          100.0       2.82          0.312     0.340
10        50          100.0       5.41          0.311     0.351
20        100         100.0       10.46         0.308     0.341

=====
PERFORMANCE ASSESSMENT
=====
[EXCELLENT] System handles load with >99% success rate
[EXCELLENT] P95 latency under 5s at max load

```

However, RAG accuracy is perform bad:

```

=====
faithfulness           : 0.834/1.000
context_recall        : 0.361/1.000
context_precision     : 0.538/1.000
answer_correctness    : 0.243/1.000
-----
OVERALL SCORE         : 0.494/1.000
=====

SCORES BY CATEGORY
=====
Category              Faithful    Recall    Precision    Correct
-----
comparison            0.700      0.000      0.000      0.113
definition            0.792      0.233      0.600      0.177
fact_retrieval        1.000      0.500      0.625      0.271
multi_hop_reasoning   0.708      0.208      0.517      0.279
numerical             0.882      0.667      0.667      0.275

Results saved to: ragas_results_20260804_013746.json
=====

RECOMMENDATIONS
=====
- LOW RECALL: Missing relevant context from documents
  → Increase top_k or improve chunking strategy
- LOW PRECISION: Retrieved chunks are not relevant
  → Improve retrieval (hybrid search, reranking)
- LOW CORRECTNESS: Answers are factually wrong
  → Improve LLM prompt or add more context

```

Metric	Current Score	Main Reason	Architectural Fix
Faithfulness	0.857	Excellent grounded generation	None needed!
Fact Retrieval	0.625	Strong BM25 + BGE-M3 matching	Working well
Comparison	0.000	Single query missing 2nd document domain	Sub-Query Decomposition
Multi-Hop	0.281	Chunks spread across multiple chapters	Query Expansion

Design Decision

Since the retrieval pipeline could now return results considerably faster, the next optimisation objective shifted from **response speed** to **retrieval accuracy**.

With GPU acceleration providing additional computational resources, there was sufficient performance budget to evaluate more advanced retrieval strategies that had previously been excluded due to latency constraints on CPU deployment.

The following improvements were therefore evaluated:

- Increasing the reranker workload by allowing more retrieved document chunks to be reranked.
- Experimenting with larger **top-k** values to improve document recall.
- Enabling **Query Expansion** to generate semantically richer search queries before document retrieval.
- Enabling **Self-RAG** to allow the system to perform an additional retrieval and self-verification step when the initial response confidence was insufficient.
- Fine-tuning retrieval parameters to balance response speed and retrieval precision.
- Evaluating whether the additional computation produced measurable improvements in RAGAS evaluation metrics.

Although **Self-RAG** and **Query Expansion** demonstrated the potential to improve retrieval accuracy, they also introduced additional LLM calls and retrieval stages, resulting in noticeably higher response latency.

These approaches were considered feasible in a GPU-enabled environment where additional computational resources were available. However, for the final CPU-based deployment, they were intentionally disabled because the project prioritised a responsive voice interaction experience. The increased latency conflicted with the primary business requirement of delivering fast, natural conversations.

Outcome

Running the system in a GPU-enabled environment demonstrated that hardware acceleration could substantially improve the responsiveness of the retrieval pipeline while also providing sufficient resources to evaluate more computationally expensive retrieval strategies.

The experiments confirmed that advanced techniques such as **Self-RAG** and **Query Expansion** can improve retrieval quality when computational resources are sufficient. However, after considering the project's deployment environment and business

requirements, the final implementation retained a lightweight retrieval pipeline to prioritise low latency and a smooth user experience over marginal improvements in retrieval accuracy.